TP5 — Introduction à la POO en PHP

Séance 1 | BTS SIO – Bloc 2

Durée : 1h30 **Notions abordées :** classe, objet, propriétés, méthodes, `$this`, instanciation

> ⚠️ **Périmètre de cette séance :** toutes les propriétés et méthodes sont déclarées en `public`. La visibilité (`private` / `protected`), les constructeurs et l'encapsulation seront abordés en séance 2.

Contexte général

Vous faites partie d'une équipe de développement qui crée un jeu de rôle (RPG) en PHP. Dans ce type de jeu, tout est un objet : les personnages, les sorts, les guildes. Votre mission est de modéliser les premières briques du jeu en appliquant les principes de la POO.

Avant de commencer : modéliser avant de coder

Pour chaque exercice, **avant d'écrire une seule ligne de PHP**, prenez 2 à 3 minutes pour répondre sur papier ou dans un commentaire :

- Quelles sont les **propriétés** de cet objet ? *(ce qu'il sait sur lui-même)*
- Quelles sont les **méthodes** de cet objet ? *(ce qu'il peut faire)*

Exercice 1 — Classe `Personnage` *(30 min)*

Contexte

Chaque joueur crée un personnage avec un nom, une classe (guerrier, mage, archer…), un niveau et des points de vie. Le personnage doit pouvoir se présenter et indiquer son état.

Modélisation attendue

Avant de coder, remplissez ce tableau :

Propriétés	Méthodes
?	?
?	?
?	?

Consignes

Créez un fichier `personnage.php`.

1. La classe

Créez une classe `Personnage` avec les propriétés publiques suivantes :

- `nom` — le nom du personnage
- `classe` — sa classe de jeu (ex : `"Guerrier"`, `"Mage"`, `"Archer"`)
- `niveau` — son niveau actuel (entier, commence à 1)
- `pointsDeVie` — ses points de vie actuels (entier)
- `pointsDeVieMax` — son maximum de points de vie (entier)

2. Les méthodes

- `sePresenter()` — retourne une ligne de présentation du personnage.
  Exemple : `"[Mage] Gandalf — Niveau 12 — PV : 45/80"`
- `estEnVie()` — retourne `true` si les points de vie sont strictement supérieurs à 0, `false` sinon.
- `soigner($points)` — ajoute des points de vie au personnage.
  - Les points de vie ne peuvent **pas dépasser** `pointsDeVieMax`.
  - Retourne un message indiquant les PV gagnés et le nouveau total.
- `monterDeNiveau()` — augmente le niveau de 1, augmente les `pointsDeVieMax` de 20, soigne complètement le personnage. Retourne un message de félicitations.

3. Les tests

Instanciez **3 personnages** avec des classes et niveaux différents. Pour chacun :

- Affichez la présentation
- Testez `soigner()` en dépassant volontairement le maximum
- Testez `monterDeNiveau()`
- Affichez la présentation finale mise à jour

Critères de réussite

- [ ] La classe est en PascalCase, les membres en camelCase
- [ ] `$this` est utilisé dans toutes les méthodes
- [ ] Le soin ne dépasse jamais `pointsDeVieMax`
- [ ] Les 3 personnages sont indépendants (modifier l'un ne touche pas les autres)

Bonus

Ajoutez une méthode `recevoirDegats($degats)` qui réduit les points de vie du personnage, en s'assurant que les PV ne descendent jamais en dessous de 0. Si le personnage tombe à 0 PV, la méthode retourne un message indiquant qu'il est hors combat.

Exercice 2 — Classe `Inventaire` *(30 min)*

Contexte

Chaque personnage possède un inventaire avec un nombre limité d'emplacements. On peut y ajouter et retirer des objets, et consulter son contenu.

Modélisation attendue

Propriétés	Méthodes
?	?

Propriétés	Méthodes
?	?

Consignes

Créez un fichier `inventaire.php`.

1. La classe

Créez une classe `Inventaire` avec les propriétés publiques :

- `proprietaire` — le nom du personnage à qui appartient l'inventaire
- `emplacementsMax` — le nombre maximum d'objets que l'inventaire peut contenir
- `objets` — un tableau (initialisé à `[]`) contenant les noms des objets

2. Les méthodes

- `ajouterObjet($nomObjet)` — ajoute un objet à l'inventaire.
  - Si l'inventaire est plein, retourne un message d'erreur sans ajouter.
  - Sinon, ajoute l'objet et retourne un message de confirmation.
- `retirerObjet($nomObjet)` — retire un objet par son nom.
  - Si l'objet n'est pas dans l'inventaire, retourne un message d'erreur.
  - Sinon, le retire et retourne un message de confirmation.
- `afficherContenu()` — affiche la liste des objets présents, les emplacements utilisés sur le total, et le poids disponible restant.
  Exemple :
  ```
  Inventaire de Gandalf (2/5 emplacements)
  1. Épée en fer
  2. Potion de soin
  ```
- `estPlein()` — retourne `true` si tous les emplacements sont occupés.
- `compterObjets()` — retourne le nombre d'objets actuellement dans l'inventaire.

3. Les tests

Créez **2 inventaires** avec des capacités différentes. Pour chaque inventaire :

- Ajoutez des objets jusqu'à atteindre la limite (testez le refus)
- Retirez un objet existant, puis tentez d'en retirer un qui n'existe pas
- Affichez le contenu à différentes étapes

Critères de réussite

- [ ] L'ajout est bloqué quand l'inventaire est plein
- [ ] Le retrait échoue proprement si l'objet est absent
- [ ] `$this` est utilisé systématiquement
- [ ] Les 2 inventaires fonctionnent de manière indépendante

Bonus

Ajoutez une méthode `contient($nomObjet)` qui retourne `true` si l'objet est déjà dans l'inventaire, `false` sinon. Utilisez cette méthode dans `ajouterObjet()` pour empêcher les doublons.

Mini-projet — Le système de Guilde *(30 min)*

Contexte

Dans le jeu, les personnages peuvent rejoindre une guilde. Une guilde a un nom, un niveau de réputation, et peut accueillir un nombre limité de membres. Les personnages et les guildes doivent interagir.

Modélisation attendue

Avant de commencer, modélisez les deux classes.

Classe `Personnage` *(version simplifiée)* :

Propriétés	Méthodes
?	?

Classe `Guilde` :

Propriétés	Méthodes
?	?

Consignes

Créez **3 fichiers** séparés.

`Personnage.php`

Classe `Personnage` avec les propriétés publiques : `nom`, `classe`, `niveau`.

Méthodes :

- `sePresenter()` — retourne une courte présentation du personnage.
- `estEligible()` — retourne `true` si le personnage est de niveau 5 ou plus (condition pour rejoindre une guilde).

`Guilde.php`

Classe `Guilde` avec les propriétés publiques : `nom`, `reputation` (entier, entre 0 et 100), `membresMax`, `membres` (tableau, initialisé à `[]`).

Méthodes :

- `recrutep($personnage)` — ajoute un objet `Personnage` au tableau `membres`.
  - Vérifie que la guilde n'est pas pleine.
  - Vérifie que le personnage est éligible (niveau ≥ 5 via `estEligible()`).
  - Retourne un message de succès ou d'erreur selon le cas.
- `expulser($nomPersonnage)` — retire de la guilde le membre dont le nom correspond.
  - Si le membre n'existe pas, retourne un message d'erreur.
  - Sinon, le retire et retourne un message de confirmation.

- `afficherMembers()` — affiche la liste des membres avec leur présentation, ainsi que le nombre de membres sur le maximum.
- `calculerNiveauMoyen()` — retourne le niveau moyen de tous les membres (0 si la guilde est vide).
- `gagnerReputation($points)` — augmente la réputation sans dépasser 100. Retourne le nouveau score.

`TestGuilde.php`

Fichier de test qui :

1. Crée **5 personnages** de niveaux variés (dont au moins 2 en dessous du niveau 5)
2. Crée **1 guilde** avec un maximum de 3 membres
3. Tente de recruter tous les personnages (certains doivent être refusés)
4. Affiche la liste des membres et le niveau moyen
5. Expulse un membre existant, puis tente d'en expulser un qui n'est pas dans la guilde
6. Fait gagner de la réputation à la guilde

Critères de réussite

- [ ] Les 3 fichiers sont créés, `require_once` est utilisé dans `TestGuilde.php`
- [ ] Les personnages sous le niveau 5 sont refusés au recrutement
- [ ] La guilde refuse les nouveaux membres quand elle est pleine
- [ ] La réputation ne dépasse pas 100
- [ ] Le code est commenté et indenté correctement

Rappel — Syntaxe de base

```php
// Déclarer une classe
class NomClasse {
    public $propriete;

    public function methode() {
        return $this->propriete;
    }
}

// Créer un objet
$objet = new NomClasse();

// Assigner des valeurs
$objet->propriete = "valeur";

// Appeler une méthode
echo $objet->methode();
```

> 💡 Si vous bloquez sur la syntaxe, reportez-vous au support de cours. Si vous bloquez sur la logique, levez la main.